

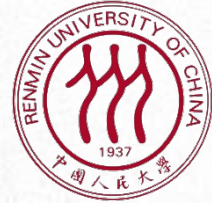
《人工智能与Python程序设计》——PyTorch基础



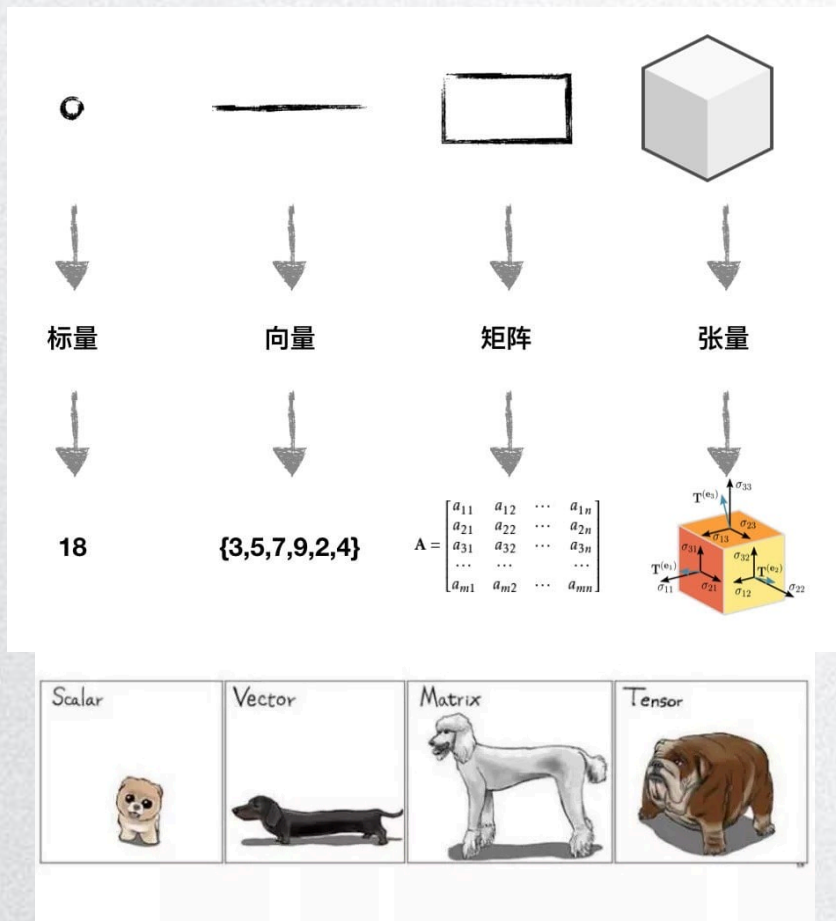
人工智能与Python程序设计 教研组



复习



- Pytorch是简单易学的人工智能平台
- 两个主要的组成部分
 - Tensor
 - AutoGrad



张量

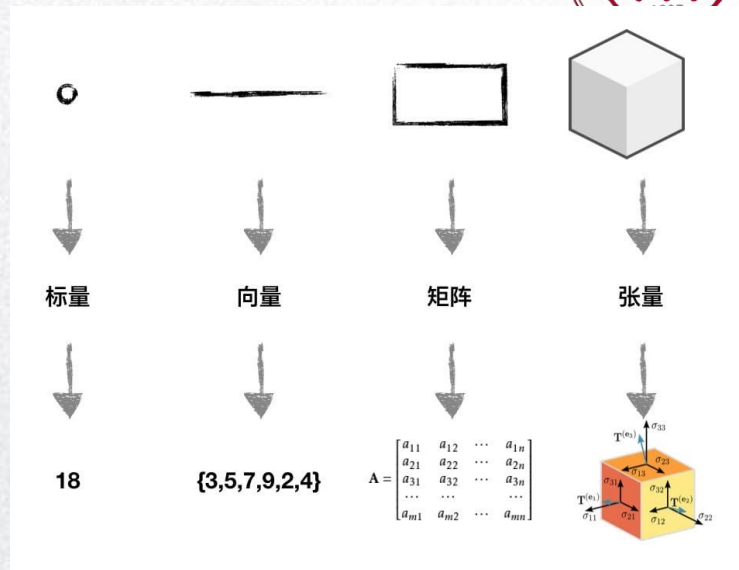
张量复习

- 张量：多维数组
- 标量—向量—矩阵——张量



PyTorch入门——Tensor

- Tensor可以认为是一个高维数组
 - 与numpy中的ndarray相似
 - 可以使用GPU加速
- Tensor的操作与numpy中的定义相似
 - 构建、初始化一个新的Tensor
 - 数学运算、线性代数运算
 - 选择、切片



PyTorch支持GPU计算

- PyTorch通过CUDA框架对GPU进行调用
- 什么是CUDA?
 - Compute Unified Device Architecture
 - NVIDIA推出的用于GPU的**并行计算**框架
 - PyTorch通过CUDA与GPU进行交互
- PyTorch中使用GPU计算
 - `t.cuda.is_available()`: 检查本机是否支持CUDA
 - `x.cuda()`, `y.cuda()`: 把Tensor x和y的内容从内存移入显存
 - 操作既可在GPU上进行

#支持GPU, 将x和y都转移到GPU中进行运算
 # 在大规模数据复杂Tensor运算时, 具有优势;
 # 小规模数据时由于具有数据转移开销, CPU会更快一点

```

#本机不支持GPU, 不会运行以下代码
if t.cuda.is_available():
    x = x.cuda() #将张量x转移到GPU中
    y = y.cuda() #将张量y转移到GPU中
    z = x + y #在GPU中运算x + y
else:
    z = x + y
print(z)

tensor([[[[0.9307, 4.5083, 1.8482, 3.4604],
          [3.0024, 1.6576, 1.1606, 1.0579],
          [3.6893, 0.3461, 0.4902, 0.5958]],

         [[1.2438, 2.6105, 2.0140, 0.6287],
          [2.3585, 3.4659, 1.9093, 2.0662],
          [3.7277, 2.7156, 3.6976, 0.8504]]]])
  
```

#如果不进行判断, `x.cuda()` 语句在没有GPU环境的情况下出错
`x = x.cuda()` #将张量x转移到GPU中
`y = y.cuda()` #将张量y转移到GPU中
`x + y` #在GPU中运算x + y

```

AssertionError                                Traceback (most recent call last)
<ipython-input-65-4a8d1eal2d04> in <module>
      1 #如果不进行判断, x.cuda() 语句在没有GPU环境的情况下出错
----> 2 x = x.cuda() #将张量x转移到GPU中
      3 y = y.cuda() #将张量y转移到GPU中
      4 x + y #在GPU中运算x + y

~\anaconda3\lib\site-packages\torch\cuda\_init_.py in _lazy_init()
    184         raise RuntimeError(
    185             "Cannot re-initialize CUDA in forked subprocess." + msg)
----> 186     _check_driver()
    187     if _cudart is None:
    188         raise AssertionError(

~\anaconda3\lib\site-packages\torch\cuda\_init_.py in _check_driver()
    59 def _check_driver():
    60     if not hasattr(torch._C, '_cuda_isDriverSufficient'):
----> 61         raise AssertionError("Torch not compiled with CUDA enabled")
    62     if not torch._C._cuda_isDriverSufficient():
    63         if torch._C._cuda_getDriverVersion() == 0:
  
```

AssertionError: Torch not compiled with CUDA enabled



GPU vs CPU

```
notebook02-modelarts-cn-north4.huaweicloud.com/modelarts/cn-north-4/hubv100/notebook/user/

jupyter Untitled Last Checkpoint: 22 minutes ago (unsaved changes)

File Edit View Insert Cell Kernel Widgets Help

In [28]: import numpy as np
import torch as t

X = t.rand(10000, 100000)
Y = t.rand(100000, 10000)
print(X.size())
print(Y.size())

torch.Size([10000, 100000])
torch.Size([100000, 10000])

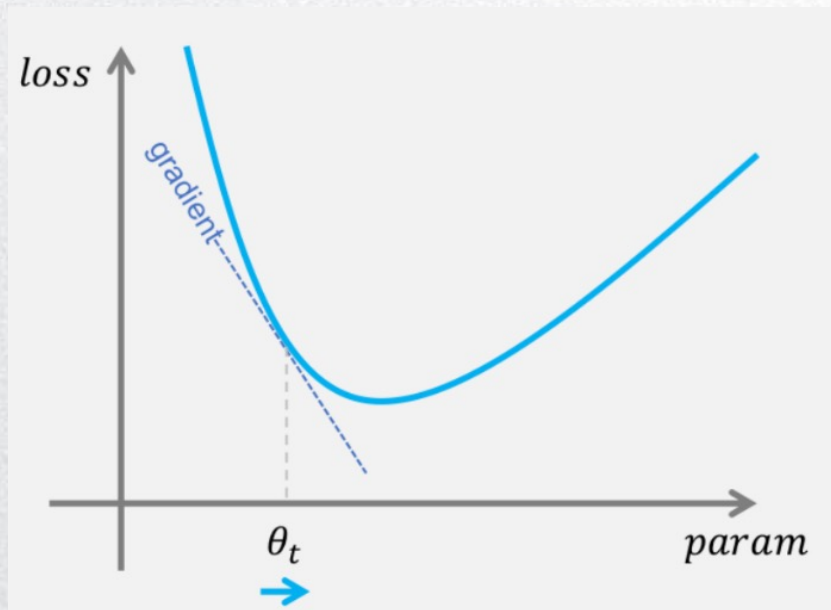
In [29]: %%time
Z = X.mm(Y)
print(Z.size())
print(type(Z))
print(Z.device)

torch.Size([10000, 10000])
<class 'torch.Tensor'>
cpu
CPU times: user 2min 33s, sys: 105 ms, total: 2min 33s
Wall time: 38.3 s

In [30]: %%time
if t.cuda.is_available():
    X = X.cuda()
    Y = Y.cuda()
    Z = X.mm(Y)
    print(Z.size())
    print(type(Z))
    print(Z.device)
else:
    print("cuda not available!")

torch.Size([10000, 10000])
<class 'torch.Tensor'>
cuda:0
CPU times: user 1.67 s, sys: 21 ms, total: 1.69 s
Wall time: 1.69 s
```

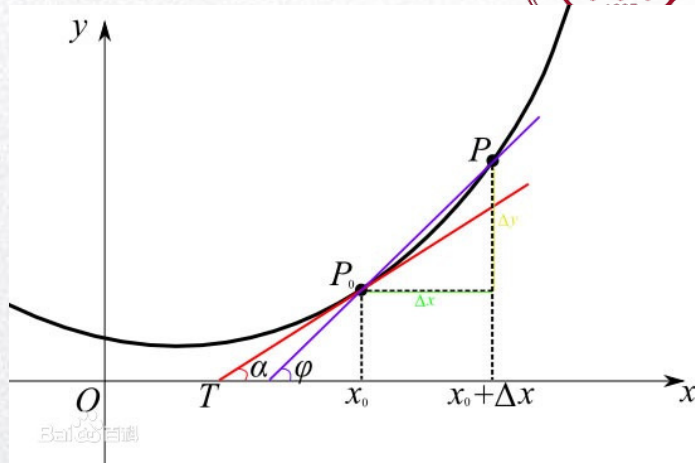
- 测试环境：华为云，1 V100 GPU
- 任务：矩阵乘法 $Z = X.mm(Y)$
 - X: 10000 * 100000
 - Y: 100000 * 10000
 - Z: 10000 * 10000
- CPU: 总耗时38.3秒
 - 结算结果Z在内存中(Z.device: cpu)
- GPU: 耗时1.69秒
 - GPU加载时间
 - X = X.cuda(): 把X移入显存
 - Y = Y.cuda(): 把Y移入显存
 - 计算时间
 - 结果矩阵Z在第一块显卡中 (cuda:0)



自动求导

函数的导数

- 函数 f 在某一点的导数描述了这个函数在这一点附近的变化率, 记为 f'
- 几何意义
 - 函数 $y = f(x)$ 在 x_0 点的导数 $f'(x_0)$ 表示函数曲线在点 $P_0(x_0, f(x_0))$ 处的切线的斜率
- 常用求导数方法
 - 1. 通过原函数求得导函数的表达式
 - 2. 把 x_0 代入导函数
- 在人工智能 (计算机) 中求导数
 - 往往只关心一个函数 (如: 损失函数) 在某些指定位置的导数的数值, 而不是导函数的表达形式
 - 问题: 是否可以跳过导函数的表达式直接求某些位置的导数值?



原函数	导函数
$y = C$ (C 为常数)	$y' = 0$
$y = a^x$	$y' = a^x \ln a$
$y = e^x$	$y' = e^x$
$y = x^n$	$y' = nx^{n-1}$
$y = \log_a x$	$y' = \frac{1}{x \ln a}$
$y = \ln x$	$y' = \frac{1}{x}$
$y = \sin x$	$y' = \cos x$
$y = \cos x$	$y' = -\sin x$

AutoGrad

- 如：函数 $y = x^2$ ，求 $x = 3$ 处的导数
 - $\frac{dy}{dx} = 2x$
 - $x = 3 \Rightarrow \frac{dy}{dx} = 2 \times 3 = 6$
- AutoGrad是PyTorch中自动求导机制，能够求得一个给定的函数在某一给定点的导数值
 - 定义输入Tensor x ，设置 `requires_grad=True`
 - 定义原函数 $y = x^2$
 - 用torch中定义的tensor运算符
 - 自动求导： `y.backward()`
 - 在 y 处求导，即 dy
 - 在 `x.grad` 处获得对 x 的导数值 dy/dx
 - 为 $x=3$ 处的导数值

```
▶ # 求函数  $y = x^2$  在  $x=3$  时的导数:  $y'(3) = dy/dx|_{x=3}$ 

x=t.tensor(3.0,requires_grad=True)
y=x.mul(x)

#判断x, y是否可以求导的
print(x.requires_grad)
print(y.requires_grad)

#求导, 通过backward函数来实现
y.backward()

#查看导数, 也即所谓的梯度
print(x.grad)

True
True
tensor(6.)
```

更多的例子

```
# 求  $y = 1/x$   
x = t.tensor(0.5, requires_grad=True)  
y = t.reciprocal(x)  
y.backward()  
print(x.grad)
```

```
x = t.tensor(1.0, requires_grad=True)  
y = t.reciprocal(x)  
y.backward()  
print(x.grad)
```

```
x = t.tensor(2, requires_grad=True)  
y = t.reciprocal(x)  
y.backward()  
print(x.grad)
```

```
tensor(-4.)  
tensor(-1.)  
tensor(-0.2500)
```

$$y = \frac{1}{x}$$

```
# 求  $y = 1/\sqrt{x}$   
x = t.tensor(0.5, requires_grad=True)  
y = t.reciprocal(t.sqrt(x))  
y.backward()  
print(x.grad)
```

```
x = t.tensor(1.0, requires_grad=True)  
y = t.reciprocal(t.sqrt(x))  
y.backward()  
print(x.grad)
```

```
x = t.tensor(2.0, requires_grad=True)  
y = t.reciprocal(t.sqrt(x))  
y.backward()  
print(x.grad)
```

```
tensor(-1.4142)  
tensor(-0.5000)  
tensor(-0.1768)
```

$$y = \frac{1}{\sqrt{x}}$$

填空题 3分



函数 $y=\max(0, x)$ 在 $x=-1$ 处的导数为 [填空1] ,
在 $x=1$ 处的导数为 [填空2] , 在 $x=2$ 处的导数为
[填空3]

正常使用填空题需3.0以上版本雨课堂

作答



练习

- 写代码求函数 $y = \max(0, x)$ 在 $x = 1$ 和 $x = -1$ 处的导数
 - $x > 0: y = x, y' = 1$
 - $x < 0: y = 0, y' = 0$
- 提示：最大值函数为`tensor.max(, )`

```
# 求  $y = \max(0, x)$  在  $x=1$  和  $x=-1$  处的导数

# 定义  $x=1.0$  和  $y=\max(0, x)$ 
x = t.tensor(1.0, requires_grad=True)
y = t.max(x, t.zeros(1, 1))

# 调用 backward 并打印导数
y.backward()
print(x.grad)

# 改变 x 后需要重新执行 y 和 backward
x = t.tensor(-1.0, requires_grad=True)
y = t.max(x, t.zeros(1, 1))
y.backward()
print(x.grad)

tensor(1.)
tensor(0.)
```



多元函数自动求导

- 类似的调用方式
- 每一个输入参数的.grad变量保存了其对应的导数值

- $f = x^2 + 2y^2 + xy$

- $\frac{df}{dx} = 2x + y$

- $\frac{df}{dy} = 4y + x$

- 在(x=1, y=1)处的导数

- $2 + 1 = 3$

- $4 * 1 + 1 = 5$

```
# f(x, y) = x^2 + 2 * y^2 + xy
```

```
# 在 (1.0, 1.0)处的导数
```

```
x = t.tensor(1.0, requires_grad=True)
```

```
y = t.tensor(1.0, requires_grad=True)
```

```
f = x.pow(2) + t.tensor(2.0).mul(y.pow(2)) + x.mul(y)
```

```
f.backward()
```

```
print(x.grad)
```

```
print(y.grad)
```

```
tensor(3.)
```

```
tensor(5.)
```



多元函数自动求导(续)

- 求 $f(x, y, z) = \ln(e^x + e^y + e^z)$ 在 $(0, 2, 5)$ 处的导数

- 思考：为什么 z 的导数最大？

- 课后练习

- 计算在其它点的导数，找出规律

```
# f(x, y, z) = ln(e^x + e^y + e^z)
# 在 (0.0, 2.0, 5.0) 处的导数
x = t.tensor(0.0, requires_grad=True)
y = t.tensor(2.0, requires_grad=True)
z = t.tensor(5.0, requires_grad=True)

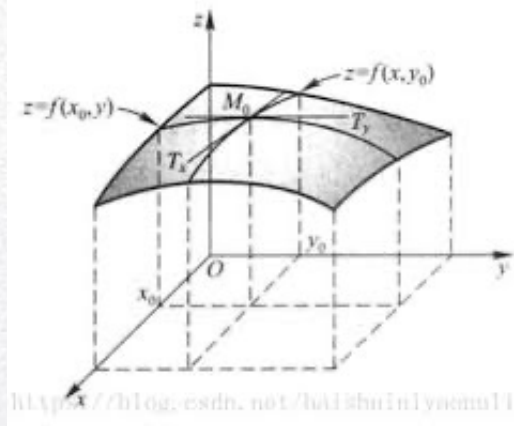
f = t.log(t.exp(x) + t.exp(y) + t.exp(z))
f.backward()
print(x.grad)
print(y.grad)
print(z.grad)

tensor(0.0064)
tensor(0.0471)
tensor(0.9465)
```


以向量为输入的函数的导数

- 函数 $f(x)$ 的输入 x 可以是向量、矩阵甚至张量(tensor)

- 如果 f 的输入是一个标量, $\frac{df}{dx}$ 也是一个标量
- 如果 f 的输入是一个 N 维向量, $\frac{df}{dx}$ 也是一个 N 维的向量
- 如果 f 的输入是一个张量, $\frac{df}{dx}$ 也是一个同样大小的张量
- 注意: 在AutoGrad中我们要求 f 的返回值为标量



- 举例: $f(\mathbf{x} = [x_1, x_2]) = x_1^2 + 2x_2^2 + 2x_1x_2 + x_1 = \mathbf{x}^T \begin{bmatrix} 1 & 1 \\ 1 & 2 \end{bmatrix} \mathbf{x} + \mathbf{x}^T \begin{bmatrix} 1 \\ 0 \end{bmatrix}$

- $\frac{df}{d\mathbf{x}} = \left[\frac{df}{dx_1}, \frac{df}{dx_2} \right]^T = [2x_1 + 2x_2 + 1, 4x_2 + 2x_1]^T$
- $\frac{df}{d\mathbf{x}} = 2 \begin{bmatrix} 1 & 1 \\ 1 & 2 \end{bmatrix} \mathbf{x} + \begin{bmatrix} 1 \\ 0 \end{bmatrix}$
- 如果 $\mathbf{x} = [1, 2]$, 导数为 $[7, 10]$



以向量为输入函数的求导

- $f(\mathbf{x}) = \mathbf{x}^T \begin{bmatrix} 1 & 1 \\ 1 & 2 \end{bmatrix} \mathbf{x} + \mathbf{x}^T \begin{bmatrix} 1 \\ 0 \end{bmatrix}$ 在(1, 2)位置的导数 (向量)

- 矩阵乘: mm()
- 矩阵转置: t()

注意: 把x和A定义为一个1*2的矩阵, x.mm()是矩阵乘法
x = t.tensor([[1.0, 2.0]], requires_grad=True)

A = t.tensor([[1.0, 1.0],
[1.0, 2.0]])

b = t.tensor([[1.0, 0.0]])

print(x, A, b)

print()

f = x.mm(A).mm(x.t()) + x.mm(b.t())

print(f)

print()

f.backward()

print(x.grad)

tensor([[1., 2.]], requires_grad=True) tensor([[1., 1.],
[1., 2.]]) tensor([[1., 0.]])

tensor([[14.]], grad_fn=<AddBackward0>)

tensor([[7., 10.]])



以向量为输入的多元函数求导

- $f(\mathbf{x}, \mathbf{y}) = \mathbf{x}^T \begin{bmatrix} 1 & 1 \\ 1 & 2 \end{bmatrix} \mathbf{y} + \mathbf{x}^T \begin{bmatrix} 1 \\ 0 \end{bmatrix}$ 在位置((1,2),(3,4))的导数 (向量)

注意: 把x和b定义为一个1*2的矩阵, x.mm()是矩阵乘法

```
x = t.tensor([[1.0, 2.0]], requires_grad=True)
```

```
y = t.tensor([[3.0, 4.0]], requires_grad=True)
```

```
A = t.tensor([[1.0, 1.0],  
              [1.0, 2.0]])
```

```
b = t.tensor([[1.0, 0.0]])
```

```
print(x, y, A, b)
```

```
print()
```

```
f = x.mm(A).mm(y.t()) + x.mm(b.t())
```

```
print(f)
```

```
print()
```

```
f.backward()
```

```
print(x.grad)
```

```
print(y.grad)
```

```
tensor([[1., 2.]], requires_grad=True) tensor([[3., 4.]], requires_grad=True) tensor([[1., 1.],  
      [1., 2.]]) tensor([[1., 0.]])
```

```
tensor([[30.]], grad_fn=<AddBackward0>)
```

```
tensor([[ 8., 11.]])
```

```
tensor([[3., 5.]])
```



基于PyTorch的线性回归梯度计算

```
import numpy as np
import torch as t
import matplotlib.pyplot as plt
```

```
def __calc_gradient(self, x, y):
```

```
    '''
```

类的方法：计算对w的梯度

```
    '''
```

```
#numpy version
```

```
    N = x.shape[0]
```

```
    diff = (x.dot(self.w) - y)
```

```
    grad = x.T.dot(diff)
```

```
    d_w = (2 * grad) / N
```

```
#torch version
```

```
    x_tensor = t.tensor(x)
```

```
    y_tensor = t.tensor(y)
```

```
    w_tensor = t.tensor(self.w, requires_grad=True)
```

```
    loss = t.sum(t.pow(x_tensor.mm(w_tensor) - y_tensor, 2)) / N
```

```
    loss.backward()
```

```
    d_w = w_tensor.grad.numpy()
```

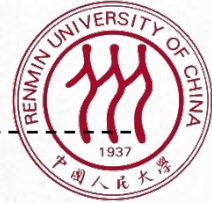
```
    return d_w
```

- 用PyTorch实现的要点

- 引入torch包
- 把numpy变量转为Tensor变量，注意只对w求导
- 用pyTorch中的方法实现损失函数
- 调用backward()
- 返回对应的梯度值

- 优点

- 不用显式求解和实现梯度公式
- 在一般情况下，梯度函数比损失函数更加难以实现



提纲

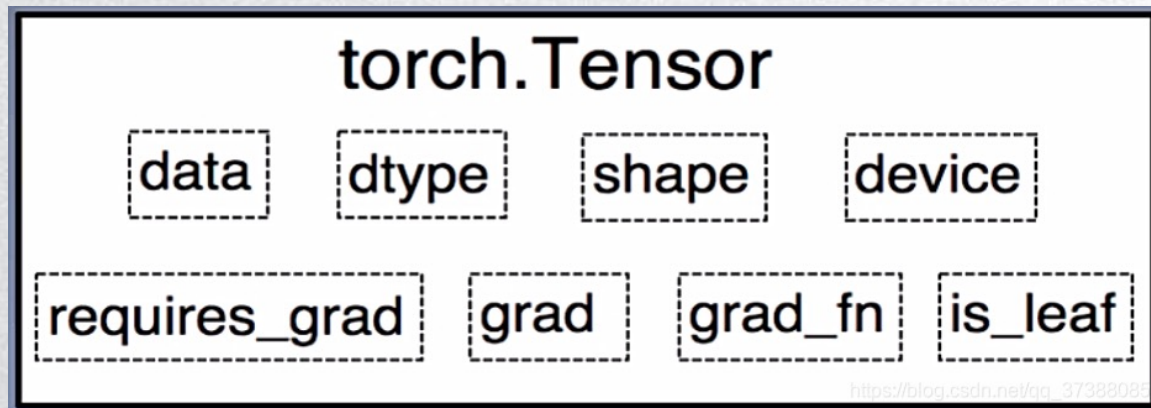


PyTorch基础

- ☐ 张量
- ☐ PyTorch nn工具箱
- ☐ PyTorch实现线性回归
- ☐ 作业: PyTorch实现逻辑回归
-

张量与自动求导

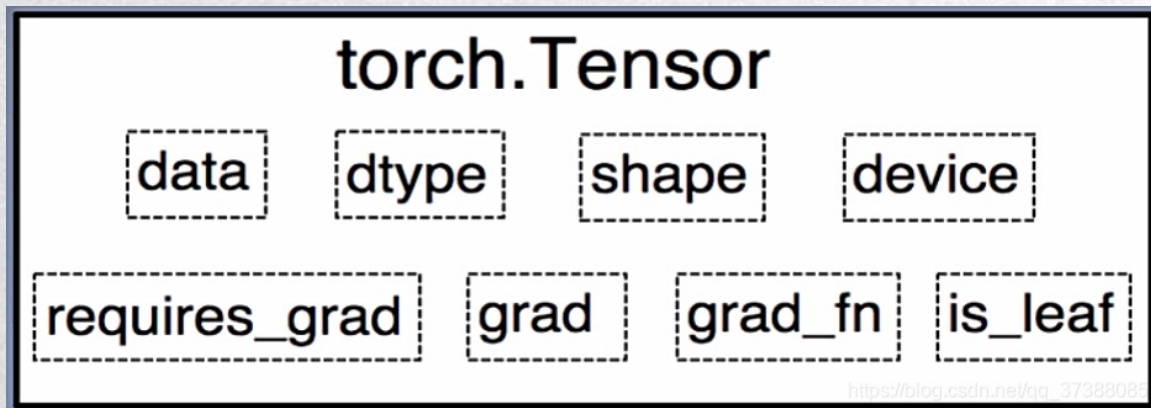
- data: 被封装的tensor
- grad: data的梯度
- grad_fn: 创建tensor的function
- requires_grad: 指示是否需要梯度
- is_leaf: 指示是否是叶子结点





张量的属性

- dtype: 张量的数据类型, 如torch.FloatTensor, torch.cuda.FloatTensor;
 - 两个数据类型之间不同之处在于后者多了一个cuda, cuda表示数据是放到GPU上的。
- shape: 张量的形状, 如 (64, 3, 224, 224)
- device: 张量所在设备, GPU/CPU





例子

```
import torch as t
A = t.empty(3, 4, requires_grad=True)
print(A.data)

tensor([[1.7266e-07, 1.2973e-11, 5.3132e-08, 2.6705e+23],
        [5.3686e-05, 4.1292e-05, 2.0431e+20, 8.5452e-07],
        [4.1429e-11, 7.1450e+31, 4.1418e-41, 0.0000e+00]])
```

```
print(A.dtype)
```

```
torch.float32
```

```
print(A.shape)
```

```
torch.Size([3, 4])
```

```
print(A.device)
```

```
cpu
```

```
print(A.requires_grad)
```

```
True
```

```
print(A.grad)
```

```
None
```

```
print(A.grad_fn)
```

```
None
```

```
print(A.is_leaf)
```

```
True
```



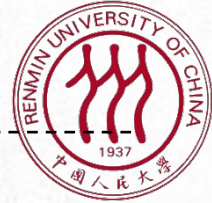


nn工具箱

- torch.nn是专门为深度学习设计的工具箱
- 主要包括：
 - 参数类 Parameter
 - 模型基类 module
 - 具有不同功能的网络层：卷积、池化、全连接、回归等
 - 非线性激活
 - 损失函数
 - 辅助函数

torch.nn

- Containers
- Convolution Layers
- Pooling layers
- Padding Layers
- Non-linear Activations (weighted sum, nonlinearity)
- Non-linear Activations (other)
- Normalization Layers
- Recurrent Layers
- Transformer Layers
- Linear Layers
- Dropout Layers
- Sparse Layers
- Distance Functions
- Loss Functions
- Vision Layers
- DataParallel Layers (multi-GPU, distributed)
- Utilities
- Quantized Functions



提纲



PyTorch基础

- ☐ 张量
- ☐ PyTorch nn工具箱
- ☐ PyTorch实现线性回归
- ☐ 作业: PyTorch实现逻辑回归
-



概念回顾

- 数据：
 - 一般为空间中点的集合，每个点以 (x_i, y_i) 的形式出现
 - 一般称 x_i 为“特征”， y_i 为对应的“真实值”或“观测值”
- **任务 (task)**：找到由 x 得到 y 的方法，即给定 x ，预测 y
 - 回归 vs 分类
- **模型 (model)**：
 - 猜想 x 到 y 的映射由 $f(x)$ 给出
 - 模型即 $f(x)$ 的具体形式，带有待定参数
- **损失函数 (loss function)**：根据已有数据，评价参数质量的指标
- **训练 (优化) (optimization)**：找到最好的 $f(x)$ 参数的过程，即最小化损失函数的过程
- 预测：给出 x ，计算 $f(x)$ 的过程
- 测试：评价预测结果

使用Tensor保存数据

继承nn.module实现模型

创建Parameter类的实例属性

通过Tensor计算得到损失函数

使用AutoGrad计算损失函数关于参数的梯度



线性回归问题回顾

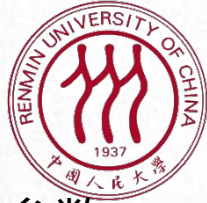
- 假设目标值与特征之间线性相关，即因变量与自变量满足一个多元一次方程。
- 训练集 $D = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_N, y_N)\}$
- 将特征 $\mathbf{x}$ 和对应的目标 y 建模为多元一次方程：

$$\hat{y} = \mathbf{x} \cdot \mathbf{w} + b$$

其中， $\mathbf{w}$ 和 b 为模型的参数。

- $\mathbf{x}$ 为行向量， $\mathbf{w}$ 为列向量
- 为了简化符号表示，在 $\mathbf{x}$ 后面加上一维值恒为1的特征， $\mathbf{x} \leftarrow [\mathbf{x}, 1]$ ，同时 $\mathbf{w}$ 增加一维，则可以把 b 融入 $\mathbf{w}$

$$\hat{y} = \mathbf{x} \cdot \mathbf{w}$$



损失函数

- 将线性预测函数式代入MSE损失函数 $L = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2$ ，将需要求解的参数 $\mathbf{w}$ 看做是损失函数 L 的自变量：

$$L(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N (\mathbf{x}_i \cdot \mathbf{w} - y_i)^2$$

- 通过最小化损失函数求解最优参数：

$$(\mathbf{w}^*) = \arg \min_{\mathbf{w}} \frac{1}{N} \sum_{i=1}^N (\mathbf{x}_i \cdot \mathbf{w} - y_i)^2$$



优化：梯度下降法

- 1. 随机选择一组初始参数($\mathbf{w}^0$)
- 2. 对于可微函数，梯度方向时函数增长速度最快的方向，梯度的反方向是函数减少最快的方向。计算损失函数在当前参数点处的梯度。
- 3. 从当前参数点向梯度相反的方向移动，移动步长为：

$$\mathbf{w}^{t+1} = \mathbf{w}^t - lr \frac{\partial L(\mathbf{w})}{\partial \mathbf{w}}(\mathbf{w}^t)$$

使用AutoGrad计算损失函数关于参数的梯度

- 4. 循环迭代步骤 3，直到前后两次迭代得到的($\mathbf{w}^t$)差值足够小，即参数基本不再变化，说明此时损失函数已经达到局部最小值。
- 5. 输出($\mathbf{w}^t$)，即为使得损失函数最小时的参数取值。



基于PyTorch的线性回归梯度计算

```
import numpy as np
import torch as t
import matplotlib.pyplot as plt
```

```
def __calc_gradient(self, x, y):
```

```
    '''
```

类的方法：计算对w的梯度

```
    '''
```

```
#numpy version
```

```
    N = x.shape[0]
```

```
    diff = (x.dot(self.w) - y)
```

```
    grad = x.T.dot(diff)
```

```
    d_w = (2 * grad) / N
```

```
#torch version
```

```
    x_tensor = t.tensor(x)
```

```
    y_tensor = t.tensor(y)
```

```
    w_tensor = t.tensor(self.w, requires_grad=True)
```

```
    loss = t.sum(t.pow(x_tensor.mm(w_tensor) - y_tensor, 2)) / N
```

```
    loss.backward()
```

```
    d_w = w_tensor.grad.numpy()
```

```
    return d_w
```

- 用PyTorch实现的要点

- 引入torch包
- 把numpy变量转为Tensor变量，注意只对w求导
- 用pyTorch中的方法实现损失函数
- 调用backward()
- 返回对应的梯度值

- 优点

- 不用显式求解和实现梯度公式
- 在一般情况下，梯度函数比损失函数更加难以实现



线性回归

- 继承nn.Module, 实现线性回归

```
class LinearRegression(nn.Module):  
    def __init__(self, in_dim): #构造函数, 需要调用nn.Module的构造函数  
        super().__init__()      #等价于nn.Module.__init__()  
        self.w=nn.Parameter(torch.randn(in_dim+1, 1))  
  
    def forward(self, x):  
        x = torch.cat([x, torch.ones((x.shape[0],1))], dim = 1)  
        x = x.matmul(self.w)  
        return x
```




Module

- Module是torch.nn 的核心数据结构，是一个抽象的概念，既可以表示神经网络的某个层(layer)，也可以表示一个包含很多层的神经网络。
- 是所有神经网络模块的基类
- Modules也可以包含其它modules。
- 成员变量：
 - training (bool)：指示这个module是训练还是测试模式
- 线性回归模型可看作是一个简单的神经网络



自定义线性回归Module

- 自定义层LinearRegression 继承nn.Module, 并且在其构造函数中调用nn.Module 的构造函数, 即

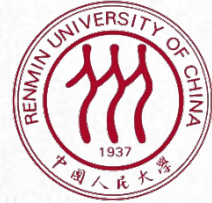
super().__init__() 或 nn.Module.__init__()

```
def __init__(self, in_dim): #构造函数, 需要调用nn.Module的构造函数  
    super().__init__()      #等价于nn.Module.__init__()
```

通过对x增加一个恒为1的特征:

```
self.w=nn.Parameter(torch.randn(in_dim+1, 1))
```

参数类



Parameter

A kind of Tensor that is to be considered a module parameter.

CLASS `torch.nn.parameter.Parameter`

[SOURCE]

A kind of Tensor that is to be considered a module parameter.

Parameters are `Tensor` subclasses, that have a very special property when used with `Module` s - when they're assigned as Module attributes they are automatically added to the list of its parameters, and will appear e.g. in `parameters()` iterator. Assigning a Tensor doesn't have such effect. This is because one might want to cache some temporary state, like last hidden state of the RNN, in the model. If there was no such class as `Parameter`, these temporaries would get registered too.

Parameters

- **data** (*Tensor*) – parameter tensor.
- **requires_grad** (*bool, optional*) – if the parameter requires gradient. See [Excluding subgraphs from backward](#) for more details. Default: `True`



参数类

- torch.nn.parameter.Parameter
- Tensor的子类，一种特殊的张量，module的参数
 - 当Parameter被指定为Module的属性时，自动注册在parameters() 迭代器
- 属性：
 - Data (Tensor)
 - requires_grad (bool, optional) : 默认是True



参数类

迭代器

迭代是Python最强大的功能之一，是访问集合元素的一种方式。

迭代器是一个可以记住遍历的位置的对象。

迭代器对象从集合的第一个元素开始访问，直到所有的元素被访问完结束。迭代器只能往前不会后退。

迭代器有两个基本的方法：**iter()** 和 **next()**。

字符串，列表或元组对象都可用于创建迭代器：

实例(Python 3.0+)

```
>>> list=[1,2,3,4]
>>> it = iter(list)    # 创建迭代器对象
>>> print (next(it))   # 输出迭代器的下一个元素
1
>>> print (next(it))
2
>>>
```




自定义线性回归Module

- 成员函数：forward()，实现前向传播过程，其输入可以是一个或多个Tensor, 对x 的任何操作也必须是Tensor 支持的操作。

forward(self, x, [y, ...])

- 定义每次调用Module实例时执行的计算，网络/层前向计算过程
- 所有子类/继承类都应重写此函数

```
def forward(self, x):  
    x = torch.cat([x, torch.ones((x.shape[0],1))], dim = 1)  
    x = x.matmul(self.w)  
    return x
```

- 无需写反向传播函数，nn.Module能够利用AutoGrad自动实现反向传播。



自定义线性回归Module

- 使用时，首先实例化一个LinearRegression类

例如：

```
def testLRmodel(in_dim, data_size = 2):  
    layer = LinearRegression(in_dim)  
    input=torch.randn(data_size,in_dim)  
    output=layer(input) #前向传播 执行forward()  
    print(output)  
    for parameter in layer.parameters():  
        print(parameter)
```

- 直观上可以将layer看成数学概念中的函数，调用model(input)即可得到input 对应的前向计算（forward）结果。

等价于layer.__call__(input).

pytorch在nn.Module中实现了__call__方法，并且在__call__方法中调用了forward函数



__call__()

- __call__()的作用是使实例能够像函数一样被调用

```
1 class X(object):
2     def __init__(self, a, b, range):
3         self.a = a
4         self.b = b
5         self.range = range
6     def __call__(self, a, b):
7         self.a = a
8         self.b = b
9         print('__call__ with ({}, {}) '.format(self.a, self.b))
```

```
1 >>> xInstance = X(1, 2, 3)
2 >>> xInstance(1,2)
3 __call__ with (1, 2)
```



自定义线性回归Module

- Module中的可学习参数可以通过成员函数parameters()返回。

```
22     for parameter in layer.parameters():  
23         print(parameter)
```

```
Parameter containing:  
tensor([[1.2220],  
        [1.0318],  
        [0.3946],  
        [0.3981]], requires_grad=True)
```




Pytorch模型训练

- 实现Linear_Model类，实现其成员函数对线性回归模型进行训练和测试
- 初始化：创建模型和优化器，初始化线性模型和优化器超参数

```
class Linear_Model():  
    def __init__(self, in_dim):  
        """  
        创建模型和优化器，初始化线性模型和优化器超参数  
        """  
  
        self.learning_rate = 0.01  
        self.epoches = 10000  
        self.model = LinearRegression(in_dim) #torch.nn.Linear(in_dim,1)  
        self.optimizer = torch.optim.SGD(self.model.parameters(), lr=self.learning_rate)  
        self.loss_function = torch.nn.MSELoss()
```

- 线性回归模型参数已在实例化LinearRegression类时用正态分布随机初始化

```
class LinearRegression(nn.Module):  
    def __init__(self, in_dim): #构造函数，需要调用nn.Module的构造函数  
        super().__init__() #等价于nn.Module.__init__()  
        self.w=nn.Parameter(torch.randn(in_dim+1, 1))
```



Pytorch优化器

- Pytoch中用来优化模型权重的类: `torch.optim.Optimizer`
- 各种常用的优化器都是Optimizer这个基类的子类
- *CLASS torch.optim.Optimizer(params, defaults)*
- 参数: params的数据类型有两种:
 - 通常把`model.parameters()`传进去



Pytorch优化器

- 随机梯度下降优化器

```
self.optimizer = torch.optim.SGD(self.model.parameters(), lr=self.learning_rate)
```

- 参数：模型的parameters()方法返回模型参数iterator
lr: 学习率

$$\mathbf{w}^{t+1} = \mathbf{w}^t - lr \left(\frac{\partial L(\mathbf{w})}{\partial \mathbf{w}} \bigg|_{\mathbf{w}^t} \right)^T$$

Pytorch优化器

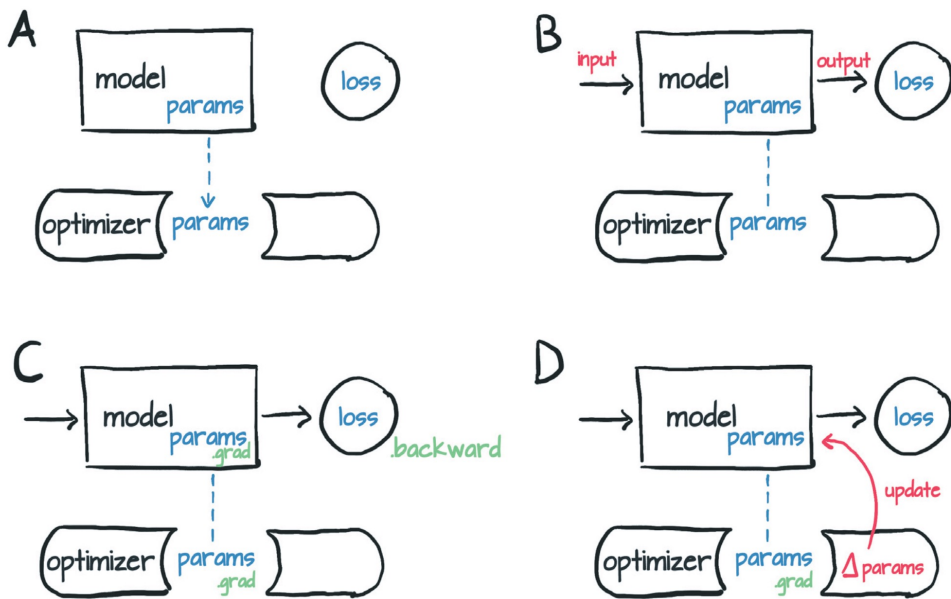
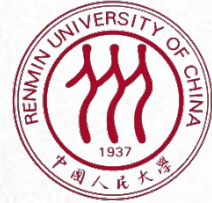


Figure 4.10 Conceptual representation of how an optimizer holds a reference to parameters (A), and after a loss is computed from inputs (B), a call to `.backward` leads to `.grad` being populated on parameter (C). At that point, the optimizer can access `.grad` and compute the parameter updates (D).



Pytorch损失函数

$$L = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2$$

- 均方误差损失 (nn.MSELoss) 实例化:

```
self.loss_function = torch.nn.MSELoss()
```

- forward: 输入两个tensor (所有样本的预测值、所有样本的真值) , 返回均方误差



Pytorch模型训练

```
def train(self, x, y):  
    """  
    训练模型并保存参数  
    输入:  
        model_save_path: saved name of model  
        x: 训练数据  
        y: 回归真值  
    返回:  
        losses: 所有迭代中损失函数值  
    """  
    losses = []  
    for epoch in range(self.epochs):  
        prediction = self.model(x)  
        loss = self.loss_function(prediction, y)  
  
        self.optimizer.zero_grad()  
        loss.backward()  
        self.optimizer.step()  
  
        losses.append(loss.item())  
  
    if epoch % 500 == 0:  
        print("epoch: {}, loss is: {}".format(epoch, loss.item()))
```

模型使用当前参数的
对训练样本进行预测

计算预测值和真值的
MSE损失

Pytorch模型训练



```
def train(self, x, y):
```

```
    """
```

训练模型并保存参数

输入:

model_save_path: saved name of model

x: 训练数据

y: 回归真值

返回:

losses: 所有迭代中损失函数值

```
    """
```

```
    losses = []
```

```
    for epoch in range(self.epochs):
```

```
        prediction = self.model(x)
```

```
        loss = self.loss_function(prediction, y)
```

```
        self.optimizer.zero_grad()
```

```
        loss.backward()
```

```
        self.optimizer.step()
```

```
        losses.append(loss.item())
```

```
    if epoch % 500 == 0:
```

```
        print("epoch: {}, loss is: {}".format(epoch, loss.item()))
```

torch.Tensor

data

dtype

shape

device

requires_grad

grad

grad_fn

is_leaf

https://blog.csdn.net/qin_3738808

pytorch中的backward()

函数的计算，梯度是
被积累的而不是被替
换掉

所有参数的梯度置0



Pytorch模型训练

```
def train(self, x, y):  
    """  
    训练模型并保存参数  
    输入:  
        model_save_path: saved name of model  
        x: 训练数据  
        y: 回归真值  
    返回:  
        losses: 所有迭代中损失函数值  
    """  
    losses = []  
    for epoch in range(self.epoches):  
        prediction = self.model(x)  
        loss = self.loss_function(prediction, y)  
  
        self.optimizer.zero_grad()  
        loss.backward()  
        self.optimizer.step()  
  
        losses.append(loss.item())  
  
        if epoch % 500 == 0:  
            print("epoch: {}, loss is: {}".format(epoch, loss.item()))
```

计算梯度



Pytorch模型训练

```
def train(self, x, y):  
    """  
    训练模型并保存参数  
    输入:  
        model_save_path: saved name of model  
        x: 训练数据  
        y: 回归真值  
    返回:  
        losses: 所有迭代中损失函数值  
    """  
    losses = []  
    for epoch in range(self.epochs):  
        prediction = self.model(x)  
        loss = self.loss_function(prediction, y)  
  
        self.optimizer.zero_grad()  
        loss.backward()  
        self.optimizer.step()  
  
        losses.append(loss.item())  
  
    if epoch % 500 == 0:  
        print("epoch: {}, loss is: {}".format(epoch, loss.item()))
```

$$\mathbf{w}^{t+1} = \mathbf{w}^t - lr \left(\frac{\partial L(\mathbf{w})}{\partial \mathbf{w}} \bigg|_{\mathbf{w}^t} \right)^T$$

所有参数更新一次



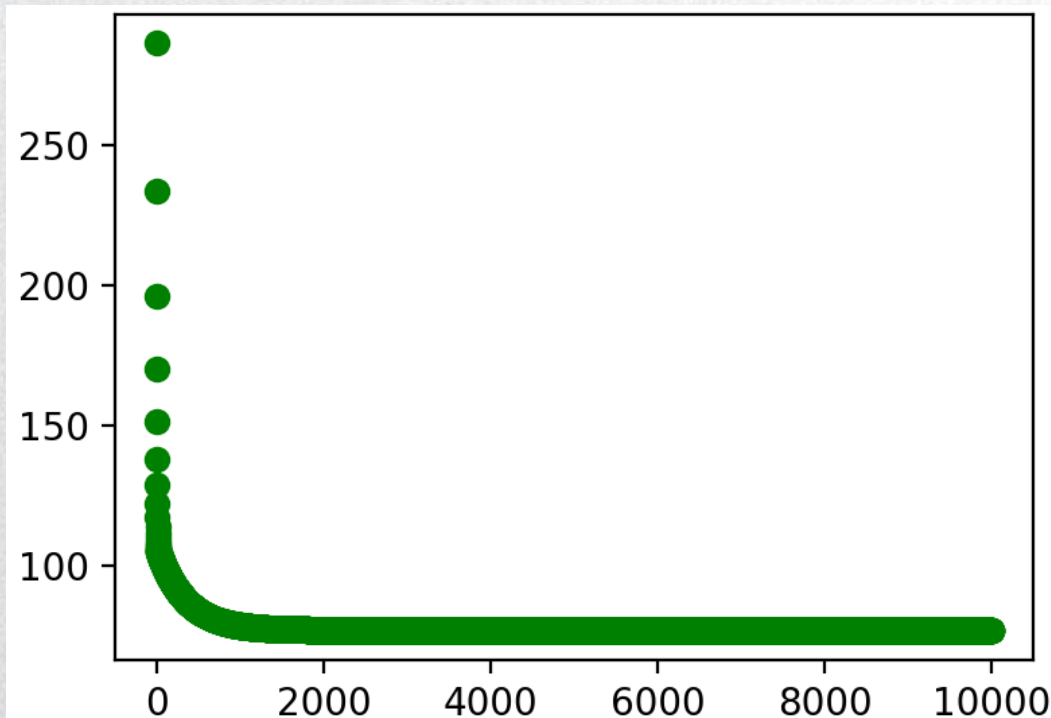
Pytorch模型训练

```
def train(self, x, y):  
    """  
    训练模型并保存参数  
    输入:  
        model_save_path: saved name of model  
        x: 训练数据  
        y: 回归真值  
    返回:  
        losses: 所有迭代中损失函数值  
    """  
    losses = []  
    for epoch in range(self.epoches):  
        prediction = self.model(x)  
        loss = self.loss_function(prediction, y)  
  
        self.optimizer.zero_grad()  
        loss.backward()  
        self.optimizer.step()  
  
        losses.append(loss.item())  
  
        if epoch % 500 == 0:  
            print("epoch: {}, loss is: {}".format(epoch, loss.item()))
```

每500轮打印一次损失值

Pytorch模型训练

- Loss随着迭代次数的变化





Pytorch模型测试

```
def test(self, x, y, if_plot = True):  
    """  
    用保存或训练好的模型做测试  
    输入:  
        model_path: 训练好的模型的保存路径, e.g., "linear.pth"  
        x: 测试数据  
        y: 测试数据的回归真值  
    返回:  
        prediction: 测试数据的预测值  
    """  
    prediction = self.model(x)  
    testMSE = self.loss_function(prediction, y)  
  
    if if_plot and x.shape[1]==1:  
        plt.figure()  
        plt.scatter(x.numpy(), y.numpy())  
        plt.plot(x.numpy(), prediction.numpy(), color="r")  
        plt.show()  
  
    return prediction, testMSE
```

用模型对测试数据进行预测

计算预测结果和测试数据真值的MSE损失



Pytorch模型测试

```
def test(self, x, y, if_plot = True):  
    """  
    用保存或训练好的模型做测试  
    输入:  
        model_path: 训练好的模型的保存路径, e.g., "linear.pth"  
        x: 测试数据  
        y: 测试数据的回归真值  
    返回:  
        prediction: 测试数据的预测值  
    """  
  
    prediction = self.model(x)  
    testMSE = self.loss_function(prediction, y)  
  
    if if_plot and x.shape[1]==1:  
        plt.figure()  
        plt.scatter(x.numpy(), y.numpy())  
        plt.plot(x.numpy(), prediction.numpy(), color="r")  
        plt.show()  
  
    return prediction, testMSE
```

用模型对测试数据进行预测

计算预测结果和测试数据真值的MSE损失



Pytorch模型测试

- `in_dim = 3`

```
w真值:tensor([[ -0.5206],  
              [-3.0018],  
              [ 1.4464]])
```

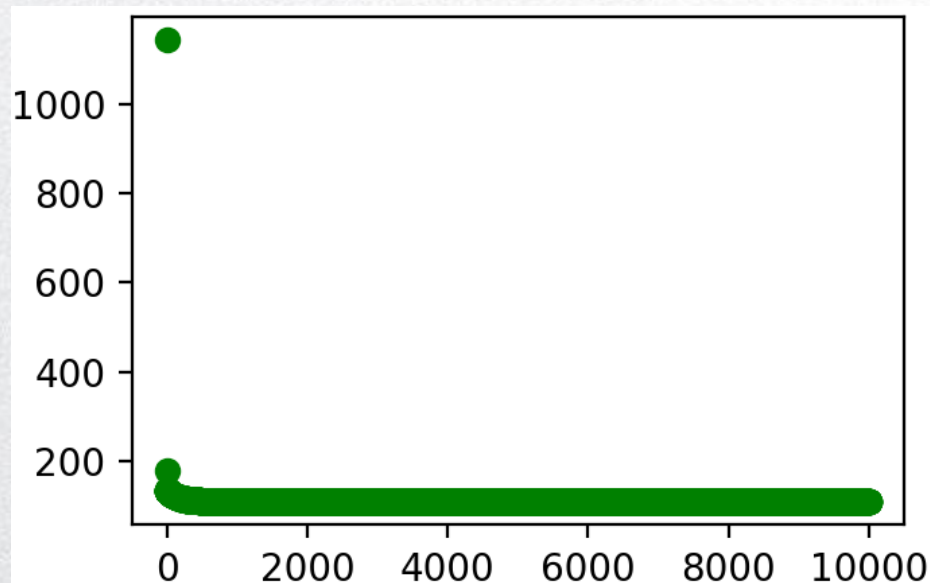
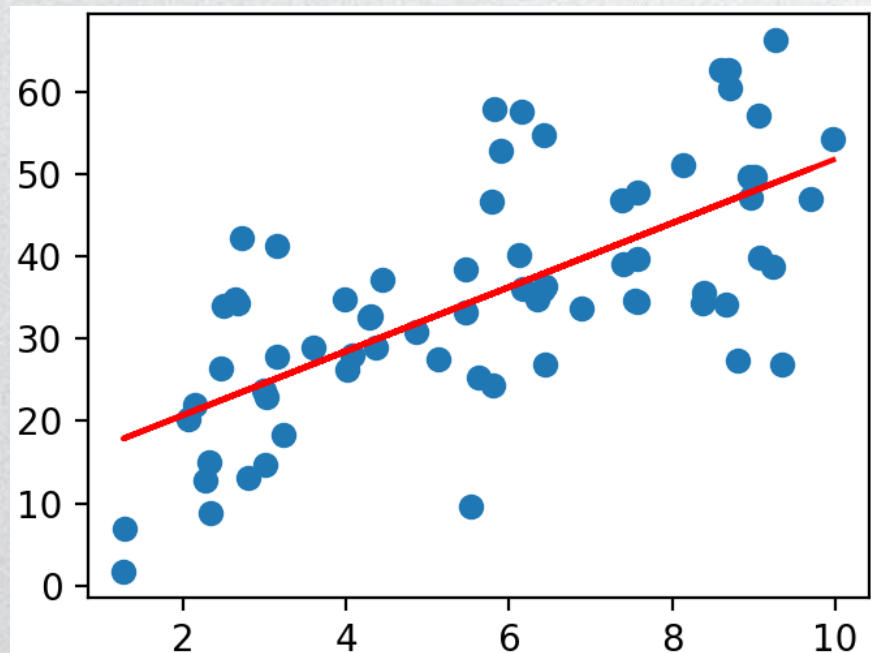
```
w Parameter containing:  
tensor([[ -0.6389],  
        [-3.8588],  
        [ 0.9694],  
        [18.1736]], requires_grad=True)
```

测试集上MSE损失值:130.86734008789062

保存的模型在测试集上MSE损失值:130.86734008789062

Pytorch模型训练测试

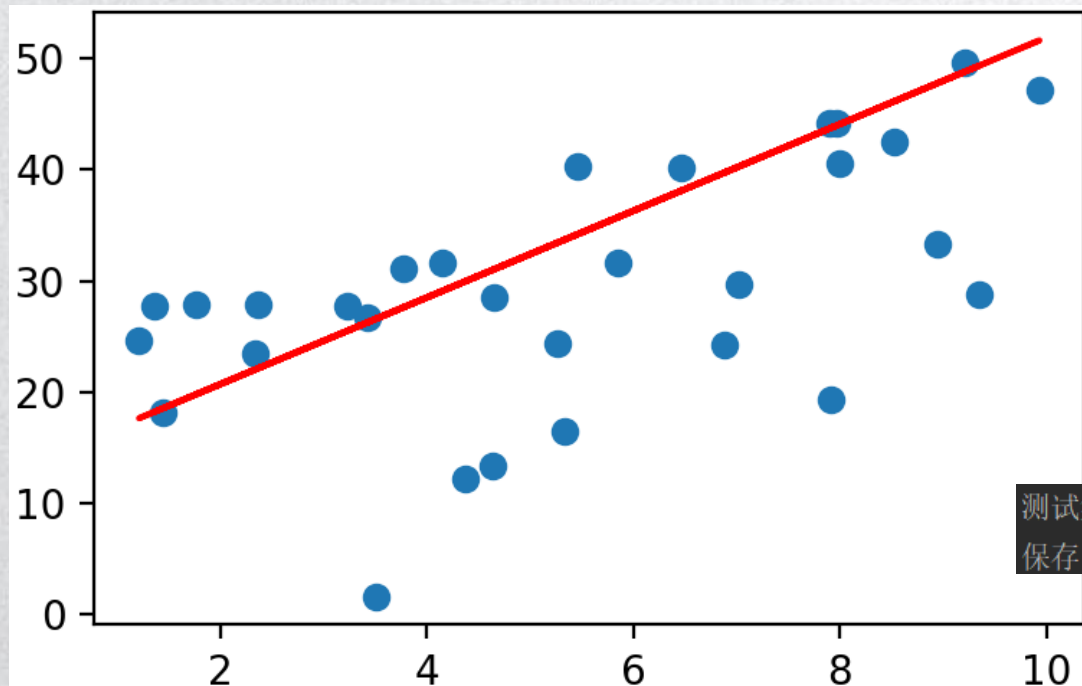
- $\text{in_dim}=1$, 训练:





Pytorch模型训练测试

- $\text{in_dim}=1$, 测试:



w真值: `tensor([[4.1975]])`

w Parameter containing:
`tensor([[3.9004],
[12.9037]], requires_grad=True)`

测试集上MSE损失值: 119.6384048461914
保存的模型在测试集上MSE损失值: 119.6384048461914



Pytorch和numpy实现线性回归对比

```
def fit(self, x, y):  
    """
```

类的方法：训练函数

参数 自变量：x

参数 因变量：y

返回每一次迭代后的损失函数

```
    """
```

```
    for i in range(self.max_iter):  
        self.__train_step(x, y)  
        y_pred = self.predict(x)  
        self.loss_arr.append(self.loss(y, y_pred))
```

```
def __train_step(self, x, y):  
    """
```

类的方法：单步迭代，即一次迭代中对梯度进行更新

```
    d_w, d_b = self.__calc_gradient(x, y)  
    self.w = self.w - self.lr * d_w  
    self.b = self.b - self.lr * d_b  
    return self.w, self.b
```

```
def __calc_gradient(self, x, y):  
    """
```

类的方法：分别计算对w和b的梯度

```
    d_w = np.mean(2 * (x * self.w + self.b - y) * x)  
    d_b = np.mean(2 * (x * self.w + self.b - y))  
    return d_w, d_b
```

PyTorch

```
def train(self, x, y):  
    """
```

训练模型并保存参数

输入：

model_save_path: saved name of model

x: 训练数据

y: 回归真值

返回：

losses: 所有迭代中损失函数值

```
    """
```

```
    losses = []
```

```
    for epoch in range(self.epochs):
```

```
        prediction = self.model(x)
```

```
        loss = self.loss_function(prediction, y)
```

```
        self.optimizer.zero_grad()
```

```
        loss.backward()
```

```
        self.optimizer.step()
```

```
        losses.append(loss.item())
```

```
    if epoch % 500 == 0:
```

```
        print("epoch: {}, loss is: {}".format(epoch, loss.item()))
```



Module

- Torch.nn几乎涵盖了所有常用的层和网络结构，比如卷积层，池化层，线性层，回归层，Transformer层等
- 通过Sequential等容器可以很容易地构造各种深度神经网络结构，就像搭积木一样
- Torch.nn同样包含了几乎所有常见的损失函数，如nn.CrossEntropyLoss, nn.MSELoss等
- 将在后续课程具体实例中介绍

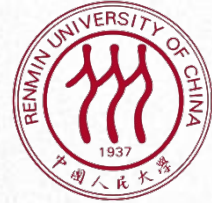


大作业3：用pytorch实现逻辑回归

- 仿照上述线性回归的代码，用pytorch实现逻辑回归
- 给定了.py文件的模板，在需要写程序的地方都有注释
- 要实现的任务包括：
 - LogisticRegression的__init__和 forward函数
 - Logistic_Model类的__init__、train和test函数
- 提交期限：2周
- 提交平台：未来学堂

```
class LogisticRegression(nn.Module):  
    def __init__(self, in_dim):  
        """  
        ***** 请在此处输入你的代码*****  
        定义模型参数w, 以及非线性sigmoid函数  
        """  
        super().__init__() # 等价于nn.Module.__init__()  
  
    def forward(self, x):  
        """
```

```
class Logistic_Model():  
    """  
    模型训练与测试类  
    """  
    def __init__(self, in_dim):...  
  
    def train(self, x, y):...  
  
    def test(self, x, y):...
```



谢谢！